Journal of King Saud University Computer and Information Sciences (2025) 37:228
https://doi.org/10.1007/s44443-025-00241-w

ORIGINAL PAPER

Machine learning-aided differential-linear attacks with applications to DES and SPECK32/64

Ze-zhou Hou[1] · Jiong-jiong Ren[1] · Shao-zhen Chen[1]

Received: 10 March 2025 / Accepted: 14 August 2025 / Published online: 24 September 2025
© The Author(s) 2025

Abstract
In CRYPTO 2019, Gohr introduced machine learning-aided differential cryptanalysis, demonstrating superior performance in key-recovery attacks compared to traditional methods. This advancement has sparked significant interest in exploring the potential of machine learning for enhancing the effectiveness and efficiency of cryptanalysis. To address these phenomena, we develop an innovative framework that integrates machine learning into differential-linear cryptanalysis and apply it to the 8-round DES, which achieves better performance than classical methods. Additionally, considering the existence of the non-trivial differentials in differential-linear distinguishers, we employ the generalized neutral bits during the generating training data phase and the key-guessing phase to improve the accuracy of neural-aided differential-linear distinguishers of SPECK. For comparison, we further perform the traditional key-recovery attacks, whose results show that machine learning-aided cryptanalysis has considerable advantages in success rate over attacks devised using pure counterparts. As the first machine learning-aided differential-linear cryptanalysis, our works not only extends the application of machine learning in cryptanalysis but also provides valuable insights into the potential of integrating deep learning for enhancing cryptanalysis.

Keywords Machine learning · Differential-linear attack · Neural distinguisher · DES · SPECK32/64

1 Introduction

Machine learning offers a wide range of applications covering many fields, including computer vision (Zou et al. 2023), speech recognition (Korvel et al. 2019), and natural language processing (Mou et al. 2016). The widespread integration of deep learning methodologies across multidisciplinary fields has catalyzed the advancement of secure and intelligent living paradigms (Roy et al. 2023; Farooqi et al. 2019; Gutub et al. 2023, 2025). And it also exhibits significant overlaps with cryptography and cybersecurity. As early as 1991, Rivest (1991) offered a survey of the relationship between cryptography and machine learning. With the development of machine learning, there are some researches about machine learning-aided cryptography including simulating ancient ciphers (Greydanus 2017), key agreement (Dong and Huang 2020) and steganalysis (Hemalatha et al. 2023; Aljarf et al. 2023, 2024).

In CRYPTO 2019, Gohr (2019) proposes a new framework utilizing machine learning to attack block ciphers. Gohr designs a training data format that is used to help neural networks capture the difference between cipher data and random data. Based on this, Gohr trains the deep residual network (denoted by ResNets, He et al. 2016) for round-reduced SPECK32/64 (Beaulieu et al. 2013). With the non-negligible accuracy, the trained neural networks are known as neural distinguishers ($\mathcal{ND}$s), which play the role of distinguisher in cryptanalysis. Besides, based on Bayesian optimization, Gohr designs a new key-recovery strategy using $\mathcal{ND}$s, reducing the time complexity of key-recovery attacks. Moreover, by adding the classical differential ($\mathcal{CD}$) before the $\mathcal{ND}$s, the distinguishers with more rounds are built and known as the hybrid distinguishers.

Considering Gohr's work that shows that machine learning-aided differential cryptanalysis has advantages over orthodox methods, more and more researchers are exploring machine learning-aided classical cryptanalysis. In EUROCRYPT

✉ Jiong-jiong Ren
jiongjiong_fun@163.com

Ze-zhou Hou
zezhouhou_hzz@163.com

Shao-zhen Chen
chenshaozhen@vip.sina.com

[1] Information Engineering University, ZhengZhou 450000, China

2021, Benamira et al. (2021) describe the relationship between Gohr's $\mathcal{ND}$s and classical truncated differentials. They show with experiments that Gohr's $\mathcal{ND}$s generally rely on the differential distribution on the ciphertext pairs, but also on the differential distribution in penultimate and antepenultimate rounds. Their work is helpful in building differential-neural distinguishers for other block ciphers. In ASIACRYPT 2022, Bao et al. (2022) focus on improving Gohr's hybrid distinguishers in various methods. In Gohr (2019), Gohr uses the neutral bits (**NB**s) to generate enough data following the same distribution, which improve the accuracy of hybrid distinguishers. Bao et al. mainly investigate the effect of neutral bits in improving the accuracy and propose the generalized **NB**s. By applying various generalized **NB**s in the $\mathcal{CD}$s, the signals from $\mathcal{ND}$s are boosted and the improved key-recovery attacks are performed on round-reduced SPECK32/64. Besides, Bao et al. also produce various neural distinguishers on round-reduced SIMON32/64 (Beaulieu et al. 2013) using multiple neural networks.

Bacuieti et al. (2022) focus on using more lightweight neural networks to train neural distinguishers. Bellini et al. (2023) offer an effective method of searching for good input differences for neural-differential distinguishers. Sarkar et al. (2024) innovatively design a novel data fusion technique to capture the features from sequential two-round ciphertext pairs, whose results improve the neural distinguishers of SPECK. There are other works on machine learning-aided differential cryptanalysis (Wang and Wang 2021; Chen et al. 2023; Lyu et al. 2022).

Most of them focus on using machine learning to assist differential cryptanalysis and show the advantages of machine learning-aided differential cryptanalysis. However, whether machine learning can be applied to other cryptanalysis is still unknown. Or, whether the advantage generally exists in the other cryptanalysis of modern ciphers?

Differential-linear cryptanalysis (Langford and Hellman 1994) is a type of combined attack that includes two well-known cryptanalysis, that is, the differential cryptanalysis and the linear cryptanalysis. Given a target block cipher, the cipher is decomposed into two sub-ciphers. For the first sub-cipher, the differential cryptanalysis is applied. And for the second sub-cipher, the linear cryptanalysis is applied. Due to the combined attacks, against some specific block ciphers, the differential-linear cryptanalysis shows better key-recovery attacks than the two basic cryptanalysis.

Considering the advantages of machine learning in differential cryptanalysis and the differential component included in differential-linear cryptanalysis, a question needs to be explored:

– Can the hybrid approach combining machine learning with the differential-linear cryptanalysis techniques exhibit superior performance in cryptanalysis tasks relative to orthodox techniques?

1.1 Main contributions

In this paper, we address this question by introducing a novel machine learning-aided differential-linear cryptanalysis framework. We demonstrate its effectiveness through a comprehensive cryptanalysis of round-reduced DES and SPECK32/64, with detailed experimental results presented in Table 1. Our works provide concrete evidence that the proposed machine learning-aided approach offers distinct advantages over classical cryptanalysis in terms of success rate and data complexity. The details of our contribution include the following.

- **Framework of machine learning-aided differential-linear cryptanalysis.** We introduce a specialized training data architecture specifically tailored for differential-linear cryptanalysis, enabling neural networks to effectively capture non-random characteristic and the differences between cipher data and random data. Employing the new training data architecture, we train the effective neural networks and further present a novel key-recovery framework that integrates these neural differential-linear distinguishers, establishing a new strategy for cryptanalytic attacks.
- **Machine learning-aided attacks on round-reduced DES.** Based on the new training data format, we train various 7-round neural distinguishers for DES. Using these neural distinguishers, we perform distinguishing attacks on 7-round DES using less plaintext than Biham et al. (2002). Moreover, neural distinguishers achieve higher success rates than the classical differential-linear distinguisher proposed in Biham et al. (2002). In addition to distinguishing attacks, using the 7-round neural distinguishers and less plaintexts than Biham et al. (2002), we build the key-recovery attacks on 8-round DES, whose success rate surpasses (Biham et al. 2002). The experiments show that it is feasible to enhance differential-linear cryptanalysis using machine learning.
- **Machine learning-aided key-recovery attacks on round-reduced SPECK32/64.** Based on the new training data format, we train various 8-round and 9-round differential-linear neural distinguishers for SPECK32/64. Using these 8-round distinguishers, we perform the 10-round key-recovery attacks by adding one free round before and one key-guessing round after 8-round distinguishers, whose results have no significant advantage in the success rate. For this, we introduce the generalized neutral bits (**NB**s) to retrain neural distinguishers, which improves the accuracy of neural distinguishers. Utilizing the improved neural distinguishers, we per-

Table 1 Summary of attacks on DES and SPECK32/64

Target Cipher	Rounds	Data (#CP)	Success Rate	Type of Attack	Ref
DES	4	-	-	key-recovery attack	(Hou et al. 2020)
	7	3590	84.13%	distinguishing attack[1]	(Biham et al. 2002)
	7	3264	85.88%	distinguishing attack[2]	Section 4
	8	14362	77.27%	key-recovery attack[1]	(Biham et al. 2002)
	8	14080	96.81%	key-recovery attack[2]	Section 4
SPECK32/64	5	< 100	-	distinguishing attack	(Zahednejad and Lyu 2022)
	10	2.56×10^6	30.5%[3]	key-recovery attack[1]	-
	10	2.56×10^6	79.6%[3]	key-recovery attack[2]	Section 5
	11	1.024×10^7	99.4%[4]	key-recovery attack[2]	Section 5
	11	$2^{13.6}$	52%	key-recovery attack[5]	(Gohr 2019)
	12	$2^{18.58}$	83%	key-recovery attack[5]	(Bao et al. 2022)
	13	2^{29}	82%	key-recovery attack[5]	(Bao et al. 2022)

[1]Classical method
[2]Machine learning-aided method
[3]The success rate is calculate over 12 bit advantages
[4]The success rate is calculate over 14 bit advantages
[5]Machine learning-aided differential cryptanalysis

form 10- and 11-round key-recovery attacks whose success rates greatly exceed the classical attacks. The first machine learning-aided differential-linear attacks on 10-, 11-round SPECK32/64 show further that the machine learning-aided method has considerable advantages in success rate over attacks devised using orthodox cryptanalysis.

1.2 Organisation of the paper

The rest of the paper is organized as follows. Section 2 introduces notations as well as basic descriptions that will be used in the rest of the paper. Section 3 proposes the framework of machine learning-aided differential-linear cryptanalysis. The practical key-recovery attack on round-reduced DES is presented in Section 4. Section 5 exhibits the application of the machine learning-aided differential-linear cryptanalysis to round-reduced SPECK32/64. Finally, the possible improvements are shown in Section 6.

2 Preliminary

2.1 Notations and conventions

Denote by $2n$ the state size in bits. For DES, n is 32. And for SPECK32/64, n is 16. Denote by (L_i, R_i) the left and right branches of a state after the encryption of i rounds. Denote by $L_i||R_i$ the concatenation of L_i and R_i. Denote by $L_i[j]$ (resp. $R_i[j]$) the j-th bit of L_i (resp. R_i) counted starting from 0, where $L_i[0]$ (resp. $R_i[0]$) is the LSB (Least Significant Bit) of L_i (resp. R_i), *i.e.* $L_i = L_i[n-1]||L_i[n-2]||...||L_i[0]$ and $R_i = R_i[n-1]||R_i[n-2]||...||R_i[0]$. Denote by $L_i[i_1, i_2, .., i_t]$ the set of i_1-th bit,...,i_t-th bit of L_i. Denote by $|x|$ the absolute value of x. Denote by $\cdot$ the bit-wise inner product. Denote by $\boxplus$ the addition modulo 2^{16}. Denote by $\oplus$ the bit-wise XOR. Denote by $\ll s$ (resp. $\gg s$) the bit-wise left (resp. right) rotation by s positions.

2.2 Brief descriptions of DES and SPECK32/64

Brief Description of DES. DES is an iterative cryptographic algorithm with Feistel structure. The encryption algorithm of DES is listed in Algorithm 1. Its 48-bit subkey K_i is generated by the 56-bit master key following its key schedule function. More details can be seen in of Standards (1977). In this paper, we omit the initial permutation IP and the final permutation IP^{-1}.

Algorithm 1 Encryption of DES.

Input $P, \{K_0, K_1, ..., K_{15}\}$
Output C
1: $(L_0, R_0) \leftarrow IP(P)$
2: **for** $i = 0$ to 15 **do**
3: $\quad L_{i+1} \leftarrow R_i$
4: $\quad R_{i+1} \leftarrow L_i \oplus F(R_i, K_i)$
5: **end for**
6: $C \leftarrow IP^{-1}(R_{16}||L_{16})$
7: return C

Brief Description of SPECK32/64. SPECK32/64 is a small member of the ARX block cipher family SPECK. The encryption algorithm of SPECK32/64 is listed in Algorithm 2. Its round function contains three operations including addition modulo 2^{16}, bit-wise rotation, and XOR. Each 16-bit subkey K_i is generated from a 64-bit master key. More details can be seen in Beaulieu et al. (2013).

Algorithm 2 Encryption of SPECK32/64.

Input $P, \{K_0, K_1, ..., K_{21}\}$
Output C
1: $(L_0, R_0) \leftarrow P$
2: **for** $i = 0$ to 21 **do**
3: $\quad L_{i+1} \leftarrow (L_i \gg 7) \boxplus R_i \oplus K_i$
4: $\quad R_{i+1} \leftarrow L_{i+1} \oplus (R_i \ll 2)$
5: **end for**
6: $C \leftarrow L_{22} || R_{22}$
7: return C

2.3 Related works of machine learning-aided cryptanalysis

As early as 1991, Rivest (1991) examined the relationship between the fields of cryptography and machine learning, emphasizing the cross-fertilization of ideas, both realized and potential. However, due to limitations in computing hardware and machine learning, relatively few researchers have focused on the intersection of cryptography and machine learning.

Motivated by the demonstrated success of machine learning in domains like image processing and signal analysis, researchers have integrated machine learning techniques into side-channel analysis (Masure et al. 2020; Picek et al. 2023). The machine learning-aided side-channel method utilizes various leakages from electronic devices, such as mechanical vibrations, electromagnetic leakage, and requires less (or no) effort to preprocess the side-channel measurements and prepare the measurements for the attack. Given these advantages, the growing popularity of deep learning for side-channel analysis in recent years is evident.

Prior to Gohr's influential research (Gohr 2019), the application of machine learning, particularly deep learning, to symmetric cryptanalysis received scant attention. His work significantly accelerated progress in this area. Gohr formulated distinguishing block cipher output pairs (from inputs with a specific difference) from random pairs as a supervised learning task, employing neural networks to enhance distinguisher accuracy. Applying this framework to SPECK32/64 (Beaulieu et al. 2013) yielded a neural distinguisher ($\mathcal{ND}$) that achieved non-negligible accuracy in identifying ciphertext pairs derived from plaintexts conforming to the input difference versus random data.

Inspired by Gohr's work, an increasing number of researchers are investigating machine learning-aided differential cryptanalysis. In EUROCRYPT 2021, Benamira et al. (2021) analyzed the internal mechanisms of Gohr's neural distinguishers, revealing that the SPECK32/64 distinguisher exploits both the differential distribution of ciphertext pairs and the corresponding distributions in the penultimate and antepenultimate rounds. In AICrypt 2023, Gohr et al. (2022) concluded that the neural distinguisher seems to be a good approximation of the differential distribution table ($\mathcal{DDT}$) for SIMON. In ASIACRYPT 2023, Bao et al. (2023) established that the neural distinguisher of SPECK can exploit the correlation between the ciphertexts' partial value, ciphertext pair's differences, and intermediate states' differences. In other words, the primary advantage of neural distinguishers over pure $\mathcal{DDT}$-based distinguishers lies in their ability to exploit differential distributions using partially known inputs to the final non-linear operation. In PST 2023, Deng et al. (2023) introduced the attention mechanism into the neural distinguisher of SPECK and analyzed the features extracted from the ciphertext by deep learning.

In addition to analyzing $\mathcal{ND}$'s internal mechanisms, researchers have pursued enhancements to its functionality. Unlike that only one pair of ciphertexts is used in one training sample in Gohr (2019), Chen et al. (2023) proposed a new training sample format by containing multiple ciphertext pairs in one single training sample. Su et al. (2020) introduced a model called the polytope differential neural distinguisher by considering multiple input differences in one training sample simultaneously. Lu et al. (2024) employed more features in one training sample by performing additional transformations on each pair of ciphertexts and proposed the neural distinguishers in related-key settings. Beyond enhancing neural distinguishers through the novel training sample format, the researchers further optimize their performance by implementing more effective neural network architectures. In ASIACRYPT 2022, Bao et al. (2022) conducted a search for a better network architecture other than the Residual Network used in Gohr's works. Employing the Squeeze-and-Excitation Network (SENet) (Hu et al. 2018), enhanced neural distinguishers for round-reduced SIMON32/64 (Beaulieu et al. 2013) are obtained. Zhang et al. (2023) introduced the Inception Module, a layer inspired by GoogLeNet's Inception module (Szegedy et al. 2015), and replaced the initial convolutional block of Gohr's distinguishers with the new module. Bellini et al. (2023) presented DbitNet, a new neural network architecture agnostic to the structure of the cipher. Using the new architecture, Bellini et al. provided new neural distinguishers for multiple primitives. While some researchers focus on developing complex neural networks to extract intricate ciphertext features, others explore the potential of simplified architectures to achieve comparable distinguishing performance with significantly reduced computational overhead (Sun et al. 2022; Shen et al. 2024).

While differential-neural cryptanalysis currently dominates machine learning-aided cryptanalysis, a parallel research stream explores deep learning-assisted enhancements for other cryptanalytic techniques. In ESORICS 2020, Hou et al. (2020) proposed the linear attack architecture based on neural network and performed the key-recovery attacks on round-reduced DES. Zahednejad and Lyu (2022) proposed the neural-based integral distinguisher scheme for the first time.

Applying the new scheme, the authors demonstrated that their neural network-based integral distinguishers can extend the number of distinguished rounds for several block ciphers. Zhang et al. (2025) employed neural networks to learn features specifically related to integral properties and obtained a key dependent integral distinguisher on 12-round SKINNY-64- 64 (Beierle et al. 2016), achieving one additional round over the previous best-known result.

Building on these advancements in the deep learning-assisted linear cryptanalysis and integral cryptanalysis, there are several questions to explore. That might include the following:

- Does there exist a viable methodology for integrating deep learning with classical differential-linear cryptanalysis?
- Can the hybrid approach combining machine learning with the differential-linear cryptanalysis techniques exhibit superior performance in cryptanalysis tasks relative to orthodox techniques?

This work primarily addresses the two research questions. By presenting the neural-based differential-linear cryptanalysis scheme, we answer the first question. Applying it to two block ciphers, we illustrate the effectiveness of our distinguisher scheme and answer the second question.

2.4 Brief introduction of classical differential-linear cryptanalysis

At CRYPTO 1994, Langford and Hellman (1994) proposed a new chosen plaintext attack on DES, which showed that a concatenation of a differential characteristic and a linear approximation is feasible.

Let $E = E_1 \circ E_0$ be an r-round block cipher which can be divided into two much simpler parts E_0 and E_1. Here, E_0 is an r_1-round block cipher and E_1 is an r_2-round block cipher. As shown in Fig. 1, for E_0, there is an r_1-round differential characteristic $\Delta_I \xrightarrow{E_0} \Delta_O$ with probability $\mathbf{Pr}_{x\in\mathbb{F}_2^n}\left(E_0(x) \oplus E_0(x \oplus \Delta_I) = \Delta_O\right) = p$, where Δ_I and Δ_O are the input and output differences of $\Delta_I \xrightarrow{E_0} \Delta_O$. And for E_1, there is an r_2-round linear approximation $\Gamma_I \xrightarrow{E_1} \Gamma_O$ with correlation $\mathbf{Cor}_{x\in\mathbb{F}_2^n}\left(\Gamma_I \cdot x \oplus \Gamma_O \cdot E_1(x)\right) = q$, where Γ_I and Γ_O are the input and output masks of $\Gamma_I \xrightarrow{E_1} \Gamma_O$.

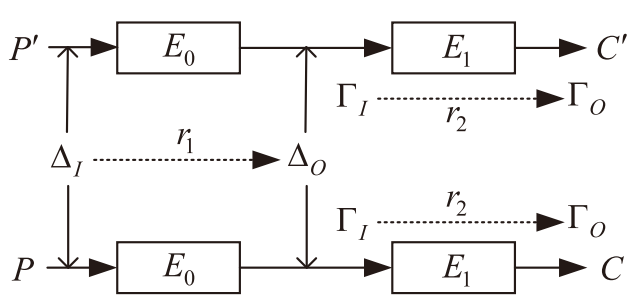


Fig. 1 The structure of a classical r-round differential-linear distinguisher

Specifically, under the assumption that $E_0(x)$ and $E_1(x)$ are independent random variables, the attacker can utilize the differential characteristic and the linear approximation to distinguish ciphers and pseudo-random permutation with correlation $\mathbf{Cor}_{P\in\mathbb{F}_2^n}\left(\Gamma_O \cdot C \oplus \Gamma_O \cdot C'\right) = pq^2$, where P and P' are encrypted r rounds to C and C' and $P \oplus P' = \Delta_I$.

As shown in Fig. 2, given an r-round differential-linear distinguisher, a key-recovery attack based on the maximum likelihood method is as follows:

Step 1. Select N plaintext pairs (P, P') with the plaintext difference $P \oplus P' = \Delta_I$.

Step 2. Request the ciphertext pairs (MC, MC') of these plaintext pairs. (Encrypt these plaintext pairs $r + l$ rounds)

Step 3. For each candidate GK_i of the last l-round key, decrypt (MC, MC') l rounds to (C, C'). Let T_i be the number of plaintexts that makes $\Gamma_O \cdot C \oplus \Gamma_O \cdot C'$ be equal to 0.

Step 4. Let T_{max} be the maximal value and T_{min} be the minimal values.

Step 5. If $|T_{max} - \frac{N}{2}| > |T_{min} - \frac{N}{2}|$, then the last l-round key is the candidate corresponding T_{max}, else the last l-round key is the candidate corresponding T_{min}.

2.5 Neutral bits and generalized neutral bits

Due to the classical differential existing in hybrid distinguishers, it is not straightforward to aggregate enough samples of the same distribution fed to neural distinguishers ($\mathcal{ND}$s). To overcome this problem, Gohr uses the neutral bits (Biham and Chen 2004) to generate enough samples of the same distribution. In Bao et al. (2022), Bao et al. give more detailed descriptions of neutral bits. In the following, the definitions of various types of neutral bits and other related elements are briefly recalled.

Definition 1 (Bao et al. 2022) Let $\delta : \Delta_{in} \xrightarrow{r\ round} \Delta_{out}$ be a differential. Let $(P, P \oplus \Delta_{in})$ be a pair of plaintexts, and let (C, C') be the corresponding ciphertext pair encrypted by r rounds. If $C \oplus C' = \Delta_{out}$, $(P, P \oplus \Delta_{in})$ is called the **conforming pair** of the differential δ.

Definition 2 (Bao et al. 2022) Let e_i be the difference with only one active bit, and let i be the location of non-zero bit. The i-th bit is a **neutral bit** of δ, if for any conforming pair $(P, P \oplus \Delta_{in})$, $(P \oplus e_i, P \oplus \Delta_{in} \oplus e_i)$ is also a conforming pair.

For the conforming pair and the neutral bits, they can be found by exhaustive search. Typically, the lower the probability of a differential, the less neutral bits. Considering that

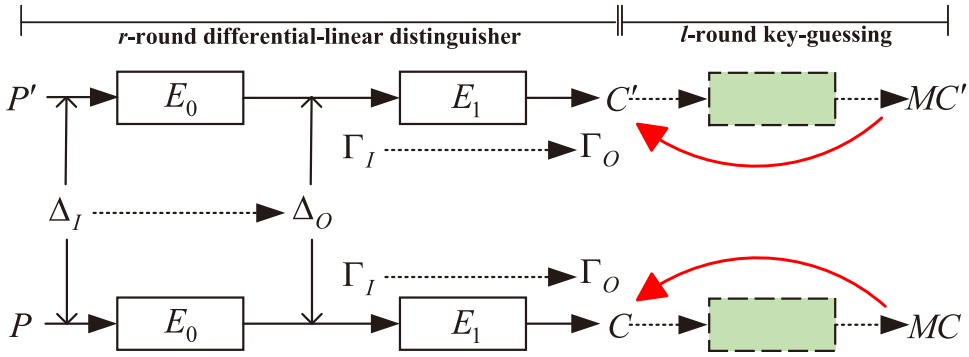


Fig. 2 Components of $(r+l)$-round differential-linear attacks

neutral bits are scarce for most differentials, the generalized neutral bits are shown in Bao et al. (2022).

Definition 3 (Bao et al. 2022) Let e_i be the difference with only one active bit, and let i be the location of non-zero bit. The i-th bit is a **probabilistic neutral bit** of δ, if $(P, P \oplus \Delta_{in})$ is a conforming pair, the event that $(P \oplus e_i, P \oplus \Delta_{in} \oplus e_i)$ is a conforming pair has a probability.

Definition 4 (Bao et al. 2022) Let $I_s = \{i_1, i_2, ..., i_s\}$ be a set of bit indices. The I_s is a **simultaneous-neutral bit-set** of δ, if for any conforming pair $(P, P \oplus \Delta_{in})$, $(P \oplus (\oplus_{i \in I_s} e_i), P \oplus \Delta_{in} \oplus (\oplus_{i \in I_s} e_i))$ is also a conforming pair, while for any subsets of I_s, the conformability of the resulted pair does not always hold.

It is difficult to exhaustively search for all simultaneous-neutral bit-sets for most of differentials. In general, the size of candidates of simultaneous-neutral bit-set is small.

3 Machine learning-aided differential-linear cryptanalysis

This section shows that the neural networks can be used in the differential-linear cryptanalysis. In the following, we design an effective method for training neural networks using the classical differential-linear distinguishers. Utilizing the trained neural networks, the attacker can distinguish cipher data with a fixed input difference from random data. Based on this, the key-recovery attack is performed.

3.1 Training neural distinguisher

3.1.1 Generation of the training data

Before training neural networks as neural distinguishers, we introduce our methods for generating the training data. It makes neural networks capture the difference between cipher data and random data. Generating enough training samples using our method, the attacker can train neural networks to distinguish cipher data from random data.

As shown in Section 2.4, $\mathbf{Pr}\left(\Gamma_O \cdot C \oplus \Gamma_O \cdot C' = 0 | P \oplus P' = \Delta_I\right) \neq \frac{1}{2}$ is an important factor of a successful differential-linear attack. And we only consider two Bernoulli distributions using neural networks. The two distributions are $Bern\left(\frac{1}{2}\right)$, denoting random data, and $Bern\left(1 - \mathbf{Pr}\left(\Gamma_O \cdot C \oplus \Gamma_O \cdot C' = 0 | P \oplus P' = \Delta_I\right)\right)$, denoting cipher data. With this, the training sample of size t is generated as follows:

1. The positive sample is generated as follows:
   (a) Randomly generate t plaintext pairs holding the plaintext difference Δ_I. These plaintext pairs are denoted by $\left\{\left(P_1, P_1'\right), ..., \left(P_t, P_t'\right)\right\}$.
   (b) Encrypt these plaintext pairs using the same master key K. The corresponding ciphertext pairs are denoted by $\left\{\left(E_K(P_1), E_K(P_1')\right), ..., \left(E_K(P_t), E_K(P_t')\right)\right\}$.
   (c) Calculate the value of x_i by $x_i = \Gamma_O \cdot E_K(P_i) \oplus \Gamma_O \cdot E_K(P_i')$, where $i \in [1, t]$.
   (d) The concatenation $x_1||...||x_t$ is treated as a single positive sample with label 1.
2. The negative sample is generated as follows:
   (a) Randomly select t independent values from the set $\{0, 1\}$, denoted as $y_1, y_2, ..., y_t$.
   (b) The concatenation $y_1||...||y_t$ is treated as a single negative sample with label 0.

$$Y\left(x_1||x_2||...||x_t\right) = \begin{cases} 1 \ if \ x_i = \Gamma_O \cdot E_K(P_i) \oplus \Gamma_O \cdot E_K(P_i'), i \in [1, t] \\ 0 \ if \ x_i \sim Bern\left(\frac{1}{2}\right), i \in [1, t] \end{cases}. \quad (1)$$

The details are shown in (1) and Fig. 3. In (1) and Fig. 3, one positive sample uses t ciphertext pairs obtained using one master key K. In other words, N master keys are used in the N positive samples. Each negative sample is generated

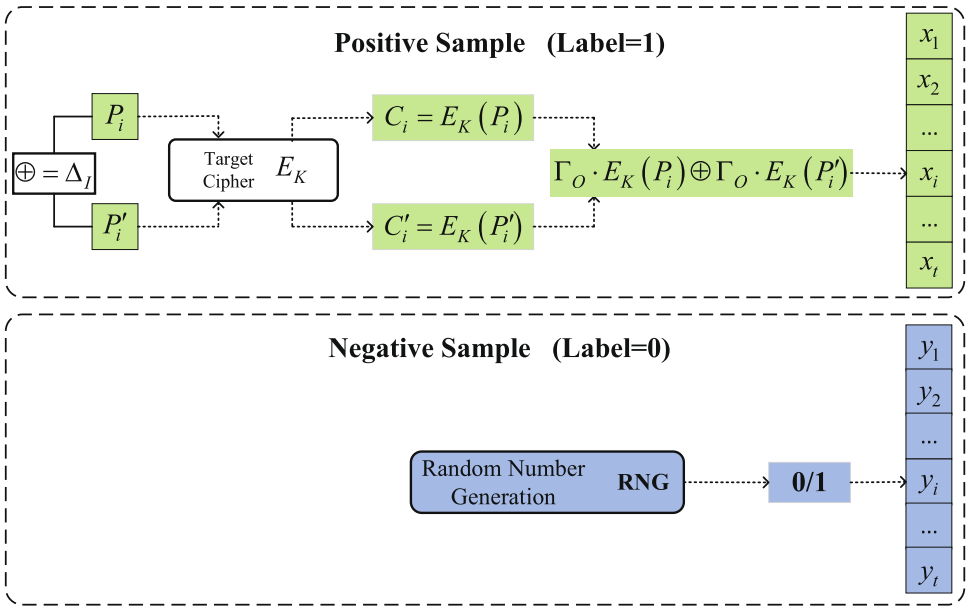


Fig. 3 Generation of training data

by the random number generator. With this method, the value of the unit in one negative sample follows $Bern\left(\frac{1}{2}\right)$. And the value of the unit in one positive sample obeys the distributions $Bern\left(1 - \mathbf{Pr}\left(\Gamma_O \cdot C \oplus \Gamma_O \cdot C' = 0 | P \oplus P' = \Delta_I\right)\right)$.

3.1.2 Neural network architecture

The neural networks used in key-recovery attacks are the same as the neural networks used in Gohr (2019), and only the input layer is modified to match the shape of the training data. In Gohr (2019), the input layer is designed to absorb the fixed-length samples (64-bit array). However, with that there are t units in the single training sample shown in Fig. 3, the input layer in our work is adjusted to absorb the training samples (t-bit array).

As shown in Fig. 4, within the **Input Layer**, the input sample is reshaped into a $1 \times t$-matrix. This matrix subsequently undergoes sequential processing by a 1D convolutional layer (`Conv1D`), batch normalization (`Batch Normalization`), and a `ReLU` activation. The resulting $1 \times t$-matrix is then propagated to the main residual tower.

In the **Iteration Layer**, the residual tower is employed five times. Each residual tower incorporates two layers, both employing t filters. Per layer, operations are applied in the sequence: 1D convolution (`Conv1D`), batch normalization (`BatchNormalization`), and `ReLU` activation. A skip connection subsequently adds the module's final activated output to its input and transmits this aggregated result to the next module.

In the **Output Layer**, the `Flatten` operation reshapes the output of the final residual tower into a vector of size t. Subsequently, two dense layers with t units each are applied. The final output is generated by a `Dense` layer with sigmoid activation.

3.1.3 Training distinguishers

The training scheme utilizes 2^{26} samples to train the neural networks, comprising equal proportions labeled 1 and 0. An additional 2^{24} samples serve as the validation set. We run the training for 50 epochs on the training set and the validation set. Operating within hardware-imposed ceilings, we set the batch size to 2^{19}. A cyclic learning rate schedule was applied, setting the learning rate l_i for i-th epoch to $l_i = 10^{-4} + \frac{(9-i) \bmod 10}{9} \times (2 \times 10^{-3} - 10^{-4})$. The Adam (Adaptive Moment Estimation) optimizer and the Mean Squared Error (MSE) loss function are employed.

Throughout the training phase, both training and validation set accuracy metrics are systematically recorded at each epoch's conclusion to comparatively assess model performance and detect potential overfitting tendencies.

With the r-round classical differential-linear distinguishers used to generate positive samples, the trained neural network is denoted as $ND_r{}^t$ if its accuracy is higher than 51%. Here, t represents the number of pairs of plaintext used in one positive sample.

3.2 Key-recovery attack architecture

As shown in Fig. 5, using the r-round neural distinguisher, the $(r + 1)$-round attacks are constructed by guessing the final subkeys. For N plaintext pairs, we decrypt the resulting ciphertext pairs under all final subkeys and rank each key can-

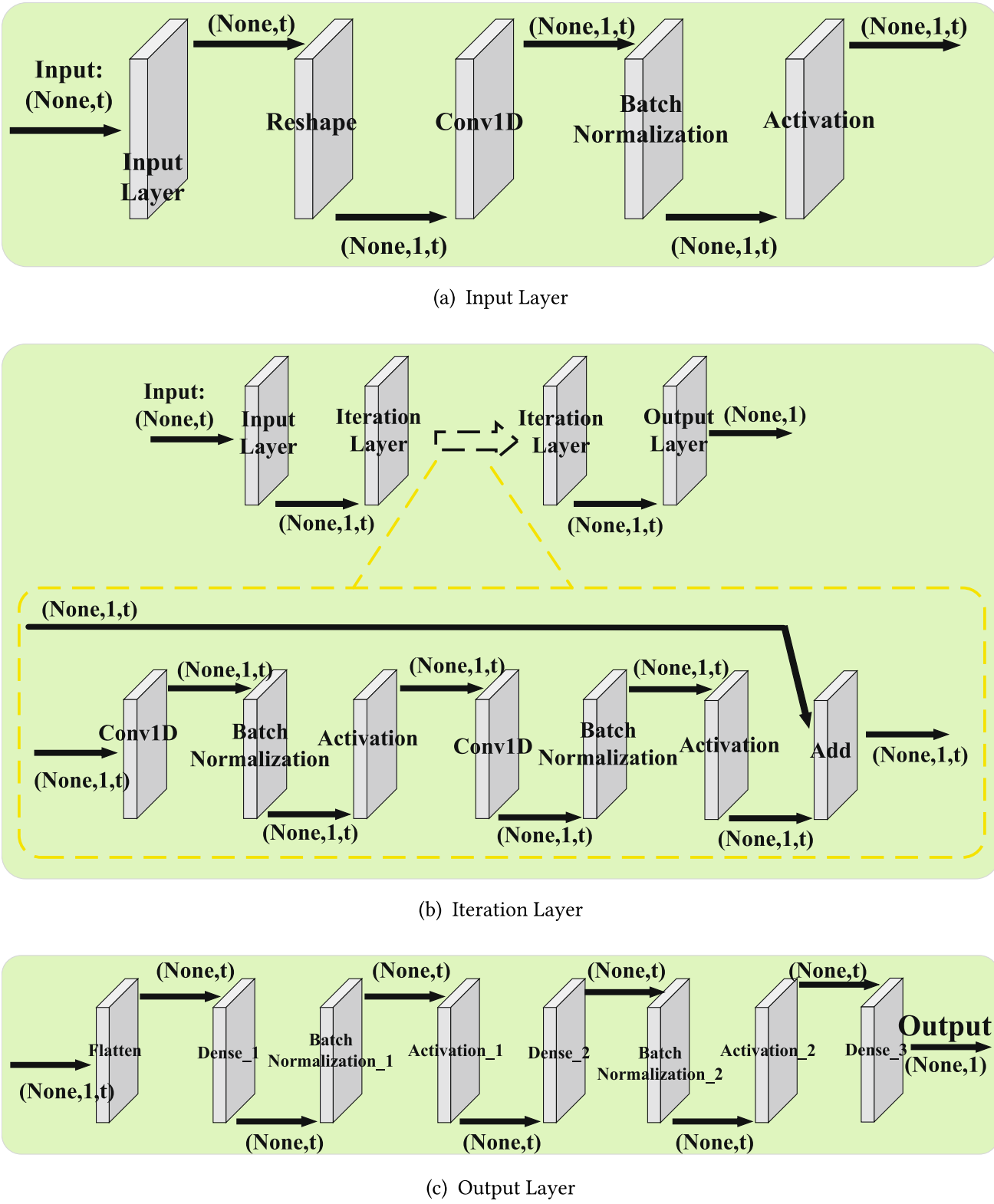


Fig. 4 Network architecture

didate using our r-round neural distinguishers. A key guess is returned if its score exceeds the scores of other candidates. The detail is as follows:

Step 1. Train the neural network and the length of a single sample is t. The trained neural network is denoted as $ND_r{}^t$.

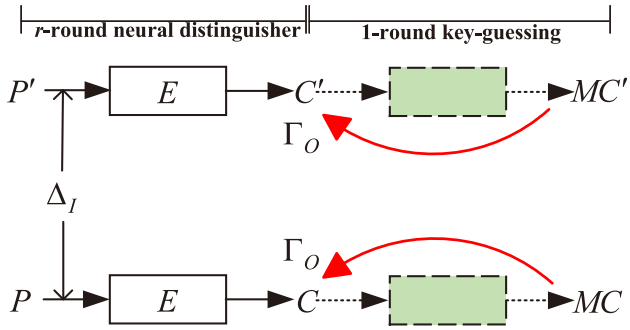


Fig. 5 (r+1)-round key-recovery attack using the r-round neural distinguisher

Step 2. Given N plaintext pairs and their corresponding ciphertext pairs, for each candidate sk_i ($i \in 1, 2, ..., l$), decrypt N ciphertext pairs and transform these ciphertext pairs into $\frac{N}{t}$ samples.

Step 3. For each candidate sk_i ($i \in 1, 2, ..., l$) and its $\frac{N}{t}$ corresponding samples, feed these samples into the trained neural network and obtain the return values ${v_j}^{sk_i}$ of ${ND_r}^t$. ${v_j}^{sk_i}$ represents the return value of the j-th sample of sk_i ($j \in 1, 2, ..., \frac{N}{t}$).

Step 4. Score each candidate sk_i ($i \in 1, 2, ..., l$), and the score of sk_i is $\sum_{j=1}^{\frac{N}{t}} \log_2 \left(\frac{{v_j}^{sk_i}}{1-{v_j}^{sk_i}} \right)$.

Step 5. Sort the candidates by the score. And the index is starting from 0.

Step 6. Return the candidate with the highest score.

We will show our machine learning-aided differential-linear cryptanalysis in further detail in Sections 4 and 5.

3.3 Framework of machine learning-aided differential-linear cryptanalysis

Given the methods for training neural distinguishers and performing the key-recovery attack, the machine learning-aided differential-linear cryptanalysis framework is shown in Fig. 6.

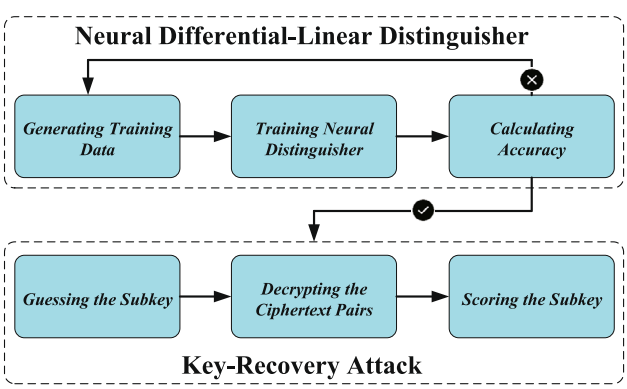


Fig. 6 Framework of machine learning-aided differential-linear cryptanalysis

As shown in Fig. 6, the machine learning-assisted differential-linear cryptanalysis framework mainly includes two phases, which are off-line training and on-line key-recovery attacks. In the off-line phase, the differential-linear neural distinguisher is trained following Section 3.1. Given the input difference and the linear masks, the enough training samples are generated to train neural networks. If the accuracy of the neural network is higher than 51%, the well-trained network is used to perform key-recovery attacks during the on-line phase. However, if the neural network is invalid, it will be essential to choose other input difference and linear masks or to generate new training samples covering more units.

Given the well-trained neural distinguisher, the key-recovery attacks are performed following Section 3.2. The encrypted ciphertext pairs are decrypted one round using the subkey candidates. These ciphertext pairs decrypted one round are used as the input of the neural distinguisher. Then these candidates are scored by the output of the distinguishers. And the candidate with the highest score is viewed as the subkey.

3.4 Experimental environment

The experiments were conducted using `Python` 3.8.19 and `Tensorflow` 2.5.0 on `Ubuntu` 20.04. Our hardware setup included Intel(R) Xeon(R) Gold 6226R processors (2.90GHz), 512GB RAM, and eight NVIDIA RTX3090 (24GB VRAM) GPUs. In fact, all computational processes were executed on a single RTX 3090 GPU (24GB VRAM), as real-time monitoring confirmed that peak VRAM utilization never exceeded 24GB during intensive training batches (size=2^{16}) nor 20GB during inference.

4 Application to DES

In Biham et al. (2002), Biham et al. improve Langford et al.'s work (Langford and Hellman 1994) with an improved differential-linear distinguisher of 7-round DES. Using the differential-linear distinguisher, the distinguishing attack on 7-round DES and the key-recovery attack on 8-round DES are performed. To compare with classical differential-linear cryptanalysis, we train the 7-round neural distinguisher of DES. Utilizing the 7-round neural distinguisher, we conduct the 7-round distinguishing attack and 8-round key-recovery attack for DES, whose results are better than Biham et al.'s results. And the experiments in round-reduced DES show that the machine learning-aided differential-linear cryptanalysis can also perform better attacks than orthodox cryptanalysis.

4.1 Training 7-round neural distinguisher

Given the classical differential-linear distinguisher with the input difference $\Delta_I = 0x0080820060000000$ and the mask $\Gamma_O = 0x2104008000008000$, we generate 2^{23} positive sam-

Table 2 Summary of accuracy of ND_7^t

ND_7^t	*Accuracy*	*TPR*	*TNR*	*FPR*	*FNR*	FLOPs	Parameter Counts
ND_7^{32}	54.29%	51.74%	56.83%	43.17%	48.26%	70.561 k	35.905 k
ND_7^{48}	55.23%	54.92%	55.54%	44.46%	45.08%	156.529 k	79.201 k
ND_7^{64}	56.01%	57.67%	54.35%	45.65%	42.33%	276.289 k	139.393 k
ND_7^{80}	56.66%	56.07%	57.25%	42.75%	43.93%	429.841 k	216.481 k
ND_7^{96}	57.32%	60.59%	54.06%	45.94%	39.41%	617.185 k	310.465 k
ND_7^{112}	57.85%	60.11%	55.60%	44.40%	39.89%	838.321 k	421.345 k
ND_7^{128}	58.46%	57.15%	59.78%	40.22%	42.85%	1093.249 k	549.121 k
ND_7^{144}	58.93%	57.17%	60.70%	39.30%	42.83%	1381.969 k	693.793 k
ND_7^{160}	59.42%	59.99%	58.85%	41.15%	40.01%	1704.481 k	855.361 k
ND_7^{176}	59.85%	58.59%	61.11%	38.89%	41.41%	2060.785 k	1033.825 k
ND_7^{192}	60.29%	61.75%	58.82%	41.18%	38.25%	2450.881 k	1229.185 k
ND_7^{208}	60.60%	63.59%	57.61%	42.39%	36.41%	2874.769 k	1441.441 k
ND_7^{224}	61.11%	64.24%	57.98%	42.02%	35.76%	3332.449 k	1670.593 k
ND_7^{240}	61.44%	66.23%	56.65%	43.35%	33.77%	3823.921 k	1916.641 k
ND_7^{256}	61.87%	61.72%	62.02%	37.98%	38.28%	4349.185 k	2179.585 k
ND_7^{272}	62.00%	69.05%	54.95%	45.05%	30.95%	4908.241 k	2459.425 k
ND_7^{288}	62.29%	65.48%	59.11%	40.89%	34.52%	5501.089 k	2756.161 k
ND_7^{304}	62.84%	65.99%	59.70%	40.30%	34.01%	6127.729 k	3069.793 k

TPR: True Positive Rate; *TNR*: True Negative Rate; *FPR*: False Positive Rate; *FNR*: False Negative Rate

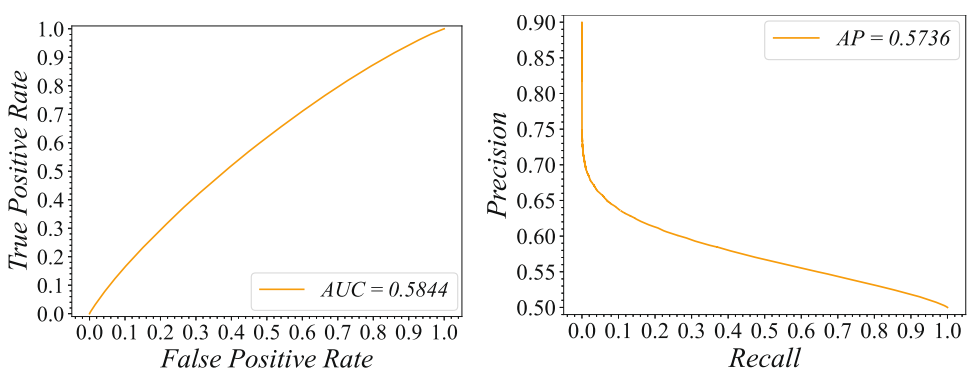
ples and 2^{23} negative samples to train the 7-round neural distinguishers ND_7^t. The length of a training sample t is found to be related to the accuracy of the neural distinguishers. In addition, we offer `FLOPs` and `Parameter Counts` as the proxy for the latency and memory usage of neural network models. We use the `Tensorflow.Keras` module `keras-flops` to evaluate the number of `FLOPs` for each distinguisher. `TensorFlow` provides a native routine `model.count_params()` for the parameter count. More details are shown in Table 2.

In addition, performance evaluations of multiple distinguishers using ROC curve and Precision-Recall curves are shown in Figs. 7, 8 and 9.

4.2 Distinguishing attack on 7-round DES

In distinguishing attack, the distinguishers are used to distinguish whether a cipher is the target cipher or a random permutation. In Biham et al. (2002), Biham et al. perform the distinguishing attack on 7-round DES using 3590 pairs of plaintexts, whose success rate is 84.13%. For comparison with Biham et al.'s distinguisher, we conduct the distinguishing attacks using N (no more than 3590) pairs of plaintexts. The distinguishing attack using neural distinguishers ND_7^t (see Table 2) is as follows:

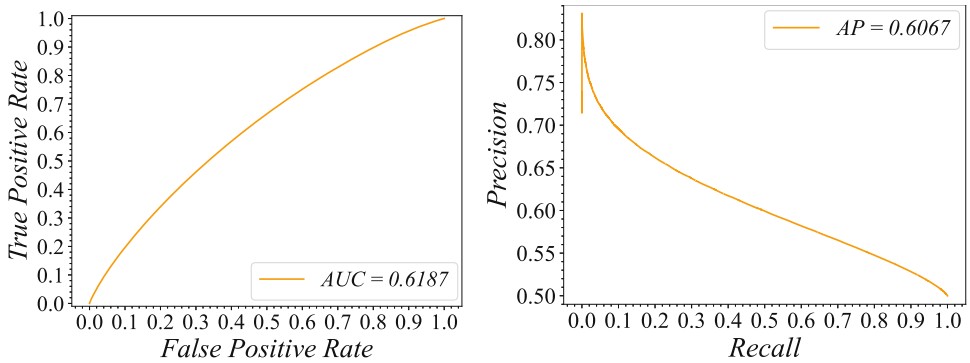
Step 1. Obtain N plaintext pairs with $\Delta_I = 0x00808200\ 60000000$ as the input difference.



(a) ROC curve

(b) Precision-Recall curve

Fig. 7 ROC curve and Precision-Recall curve of ND_7^{64}



(a) ROC curve (b) Precision-Recall curve

Fig. 8 ROC curve and Precision-Recall curve of $ND_7{}^{128}$

Step 2. Input these plaintext pairs into a **Black Box** (7-round DES or random permutation), request the outputs of the **Black Box**. These outputs are denoted as O_C.

Step 3. Transform these data in O_C to $\frac{N}{t}$ samples.

Step 4. Input $\frac{N}{t}$ samples into $ND_7{}^t$ and v_i is the response of the i-th sample.

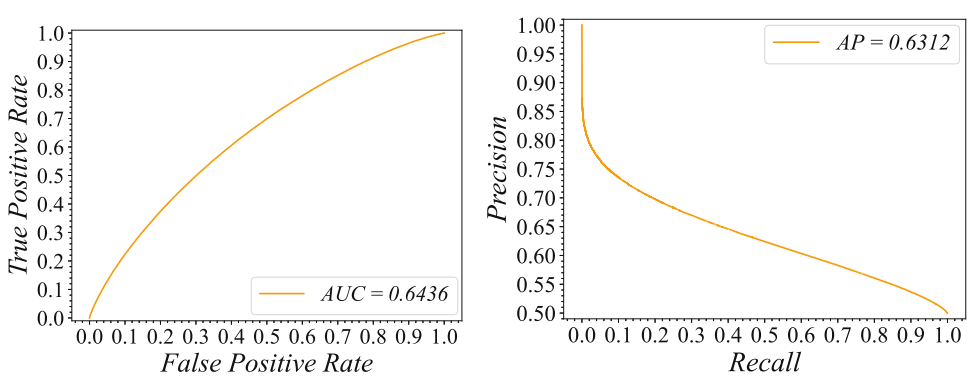
Step 5. If $v = \sum_{i=1}^{\frac{N}{t}} \log_2\left(\frac{v_i}{1-v_i}\right) > 0$, then guess the **Black Box** is 7-round DES; otherwise, guess it is an random permutation.

Given N ciphertext pairs encrypted by 7-round DES, there are $\frac{N}{t}$ positive samples for the neural distinguisher $ND_7{}^t$. Using these positive samples as the input of $ND_7{}^t$, there are $\frac{N}{t}$ return values of the distinguishers. The combined response of these samples is calculated by $v = \sum_{i=1}^{\frac{N}{t}} \log_2\left(\frac{v_i}{1-v_i}\right)$. Here, the v_i denotes the i-th response value of the i-th positive sample. If $v > 0$, it is viewed as one successful distinguishing attack.

Similarly, given N random pairs processed by the random permutation, there are $\frac{N}{t}$ negative samples for the neural distinguisher $ND_7{}^t$. Using these negative samples as the input of $ND_7{}^t$, there are $\frac{N}{t}$ return values of the distinguishers. The combined response of these samples is calculated by $v = \sum_{i=1}^{\frac{N}{t}} \log_2\left(\frac{v_i}{1-v_i}\right)$. Here, the v_i denotes the i-th response value of the i-th negative sample. If $v < 0$, it is viewed as one successful distinguishing attack.

Performing n distinguishing attacks using $n \times N$ ciphertext pairs encrypted by 7-round DES, the number of successful attacks is denoted as n_{DES}. Similarly, performing n distinguishing attacks using $n \times N$ random pairs processed by the random permutation, the number of successful attacks is denoted as n_{per}. Thus, the success rate of the distinguishing attack using $ND_7{}^t$ is calculated by $\frac{(n_{\text{DES}}+n_{per})}{2\times n}$.

We perform 10^4 distinguishing attacks for each of neural distinguisher. Half of them uses the 7-round DES as the **Black Box**. The results using different neural distinguishers are summarized in Table 3. With that the success rate of the distinguishing attack is 84.13% using 3590 plaintext pairs and the classical distinguisher, we perform the distinguishing attack using fewer plaintext pairs. Taking the results using $ND_7{}^{128}$ as an example, there are $\frac{3072}{128} = 24$ positive samples using 3072 ciphertext pairs. Performing the distinguishing attack with the 24 positive samples, the attack is viewed as a successful attack if the combined response of the 24 samples



(a) ROC curve (b) Precision-Recall curve

Fig. 9 ROC curve and Precision-Recall curve of $ND_7{}^{192}$

Table 3 Summary of distinguishing attacks of DES

$ND_7{}^t$	N	Success Rate	$ND_7{}^t$	N	Success Rate
$ND_7{}^{64}$	3392	85.41%	$ND_7{}^{80}$	3440	85.40%
$ND_7{}^{128}$	3072	85.20%	$ND_7{}^{144}$	3168	85.51%
$ND_7{}^{176}$	3344	85.75%	$ND_7{}^{192}$	3264	85.88%

is higher 0. Repeating the attacks 0.5×10^4 times, there are $n_{\text{DES}} = 4297$ successful trails. On the other hand, for the distinguishing attacks using 24 negative samples, there are $n_{per} = 4223$ successful trails when performing 0.5×10^4 attacks. Thus, the success rate of the distinguishing attack using $ND_7{}^{128}$ is $\frac{(4297+4223)}{10^4} = 0.8520$.

As shown in Table 3, the higher success rates of the distinguishing attacks are obtained by using multiple neural distinguishers covering multiple size of the length of the training samples. It is found that using neural distinguishers can perform better distinguishing attacks with fewer plaintext pairs and higher success rate than using classical differential-linear distinguishers.

4.3 Key-recovery attack on 8-round DES

In Biham et al. (2002), Biham et al. perform the 8-round key-recovery attack by adding 1 round after the 7-round differential-linear distinguisher. It recovers the 6 bits of the last subkey. The success rate is 77.27% using 14362 plaintext pairs. In this section, we also perform the key-recovery attacks on 8-round DES using no more than 14362 plaintext pairs by adding 1 round after our neural distinguishers $ND_7{}^t$, as follows:

Step 1. Given N ciphertext pairs encrypted by 8-round DES with the input difference $\Delta_I = 0x0080820060000000$, decrypt these ciphertext pairs 1 round using the key candidate[1] k_i. Here, k_i denotes the i-th candidate.

Step 2. Transform these ciphertext pairs decrypted 1 round by the key candidate into $\frac{N}{t}$ samples, where t is the length of the training sample in training $ND_7{}^t$.

Step 3. Feed these samples into $ND_7{}^t$ and $v_j^{k_i}$ is the response of j-the sample.

Step 4. The key candidate k_i is scored by $score_{k_i} = \sum_{j=1}^{\frac{N}{t}} \log_2 \left(\frac{v_j^{k_i}}{1-v_j^{k_i}} \right)$.

Step 5. The real key is the candidate with the highest score.

Following the above procedures, the key-recovery attacks on 8-round DES are performed. Given the encryption of 8-round DES with the unknown key, the N ciphertext pairs in the **Step 1** are encrypted using the unknown key. Running the **Step 1**-**Step 5** in order, the output of the **Step 5** is viewed as the best candidate for the last subkey of the unknown key. If the best candidate is equal to the real last subkey, the attack is viewed as one successful key-recovery attack. Actually, with non-zero bits in the mask $\Gamma_O = 0x2104008000008000$, the above procedures are used to recover the value of $K_7[47, 46, 45, 44, 43, 42]$[2]. If the value of $K_7[47, 46, 45, 44, 43, 42]$ is correctly guessed, we count a key guess as successful.

Using $N = 14080$ plaintext pairs and $ND_7{}^{128}$, the success rate is 96.811% among 10^5 trials of the machine learning-aided differential-linear attacks. For comparison, the 8-round classical attack (Biham et al. 2002) using the 7-round classical distinguisher is expected to succeed with 14362 plaintext pairs in 77.27%. The result shows that machine learning-aided differential-linear cryptanalysis has a distinct advantage over classical differential-linear cryptanalysis in the success rate and data complexity.

4.4 Complexity of the attack

To further show the performance of the neural distinguisher, we compare the machine learning-aided cryptanalysis with the classical cryptanalysis by the complexity of the attack.

4.4.1 Complexity of the distinguishing attack

As shown in Fig. 10, performing the distinguishing attack using the classical method mainly includes two steps. Given the ciphertext pair (C, C'), the attacker computes the value of $\Gamma_O \cdot C \oplus \Gamma_O \cdot C'$ in the first step. Then, the number of ciphertext pairs satisfying $\Gamma_O \cdot C \oplus \Gamma_O \cdot C' = 0$ is computed.

Different from the classical method, for the machine learning-aided method, there are different processes after computing the parity of the bits masked. In the machine learning-aided method, the multiple values of $\Gamma_O \cdot C \oplus \Gamma_O \cdot C'$ are transformed into the input of the neural distinguisher in the second step (refer to the **Step 3-4** in Section 4.2). And the combined response is calculated in the third step (refer to the **Step 5** in Section 4.2). When a fast graphics card (e.g. NVIDIA RTX3090) is used, the time of calculating the input samples using the neural distinguisher is negligible. Taking the distinguishing attacks using $ND_7{}^{128}$ and 3072 ciphertext pairs (shown in Table 3) as an example, the average GPU time (inclusive of host-device data transfer latency) per trial on our server is 0.007 seconds. In other words, the total time of performing the distinguishing attack using the neural distinguish mainly focuses on the time of computing the values of $\Gamma_O \cdot C \oplus \Gamma_O \cdot C'$ for multiple ciphertext pairs.

[1] Observing the mask $\Gamma_O = 0x2104008000008000$, there are 2^6 candidates in total.

[2] These 6 bits of the last subkey is related to the input of the first S-box in the last-round encryption function.

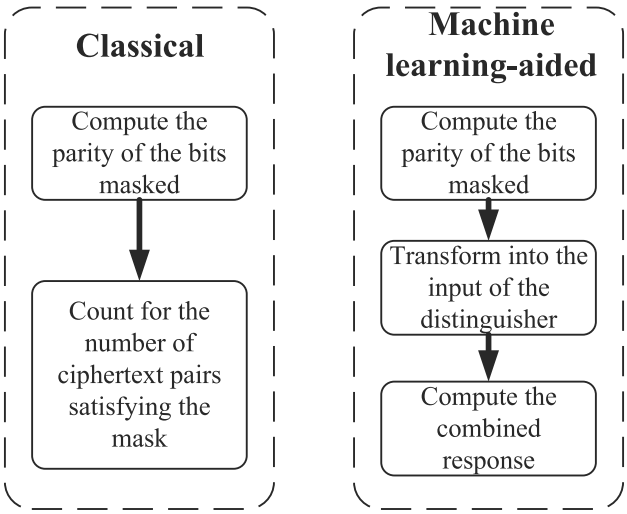


Fig. 10 The components of performing the distinguishing attack

Consequently, the time complexity of using the machine learning-aided method is only a little higher than that of using the classical method, if the same number of ciphertext pairs is used.

However, as shown in Section 4.2, there are fewer ciphertext pairs used to perform the distinguishing attack with the higher success rate. Accordingly, there is a better distinguishing attack using neural distinguishers.

To empirically validate that the novel framework imposes no prohibitive computational overhead, we systematically monitor the utilization of memory and GPU memory during distinguishing attack execution. Employing `Python`-based memory profiling (`Python` library `memory_profiler`), we monitor peak memory utilization and empirically documented a maximum allocation of 2391.0MB throughout all experimental operations. The predominant share of allocated memory resources is used to store the ciphertexts and the output of the distinguishers. Using the system module `nvidia-smi`, it is recorded that the maximum GPU memory allocation is 1180MB.

4.4.2 Complexity of the key-recovery attack

As shown in Fig. 11, in key-recovery attacks, partial decryption of ciphertext pairs precedes the computation of the masked bits' parity. For the machine learning-aided method, the total time is influenced by the time of decrypting the ciphertext pairs and the GPU time. Running 10^5 key-recovery attacks, the average attack time per trial on our server is 3.637 seconds. And the average GPU time (inclusive of host-device data transfer latency) per trial on our server is 1.885 seconds. Similarly, we systematically monitor the utilization of memory and GPU memory during the execution of the key recovery attack. We monitor peak memory utilization and empirically documented a maximum allocation of 2473.2MB throughout all experimental operations. It is recorded that the maximum GPU memory allocation is 1244MB.

5 Application to SPECK32/64

In Niu et al. (2022), Niu et al. present the 8-round differential-linear distinguisher with the input difference $\Delta_I = 0x02110a04$ and the bit mask $\Gamma_O = 0x00080008$, and the 9-round distinguisher with the input difference $\Delta_I = 0x02110a04$ and the bit mask $\Gamma_O = 0x58204020$. In this section, we perform the 10-round machine learning-aided attack based on an 8-round neural distinguisher. However, its success rate is lower than that of the classical differential-linear attack.

An improved attack is performed by using the generalized neutral bits in the training phase and the key-guessing phase. In training neural networks, we use **NB**s to generate positive samples. In key-recovery attacks, we construct plaintext sets using **NB**s. With this, an improved attack is performed and its success rate far exceeds that of classical differential-linear attack. Moreover, the 11-round machine learning-aided attack is performed employing **NB**s and 9-round neural distinguisher.

5.1 Key recovery attack on 10-round Speck32/64

5.1.1 Basic attack

We train the neural network ${ND_8}^{256}$ using $\Delta_I = 0x02110a04$ as the input difference and $\Gamma_O = 0x00080008$ as the bit mask. Its accuracy is about 52.7%.

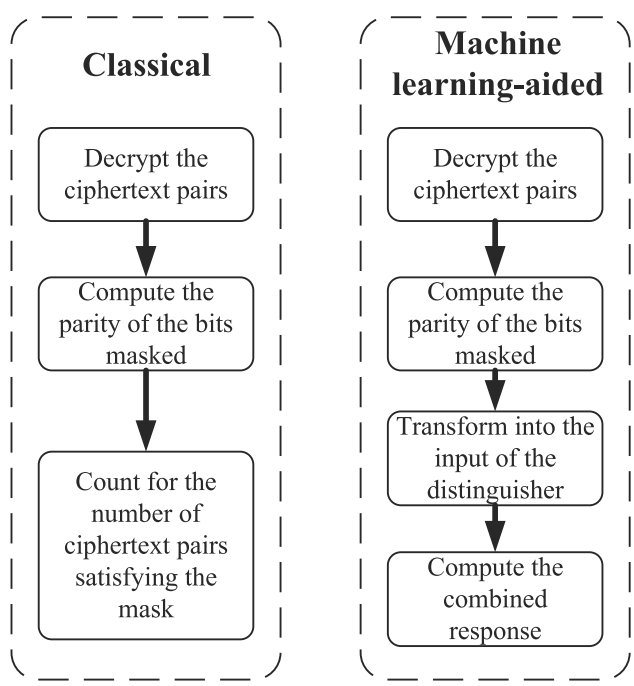


Fig. 11 The components of performing the key-recovery attack

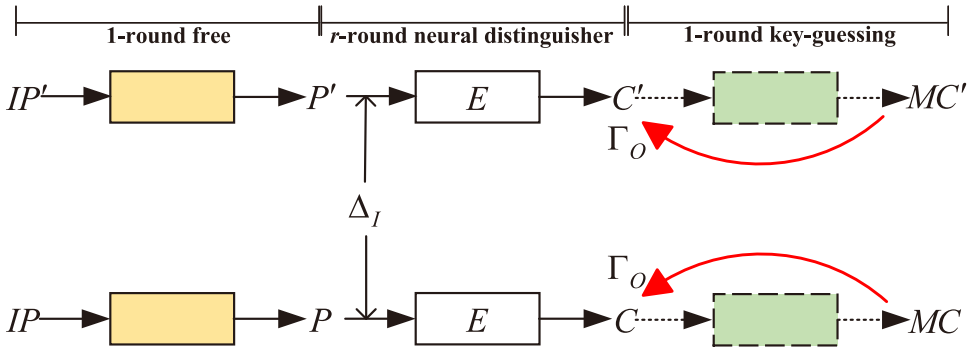


Fig. 12 $(1 + r + 1)$-round key-recovery attack of SPECK

As shown in Fig. 12, the configuration of 10-round attacks is 1+8+1, *i.e.*, a free prepended invert round, an 8-round neural distinguisher, and a key guessing (last) round. Considering the locations of nonzero bits of Γ_O, we only guess 13 bits of the final subkey ($K_7[0, 1, ..., 12]$). Then we perform the key-recovery attacks using $N = 2.56 \times 10^6$ plaintext pairs and $ND_8{}^{256}$. With 10^3 trials, the result is shown in the second row (*Basic Attack*) of Table 4. And we perform 10^3 classical attacks on 10-round SPECK32/64 using the 8-round distinguisher $\Delta_I = 0x02110a04 \rightarrow \Gamma_O = 0x00080008$ and 2.56×10^6 plaintext pairs, whose result is shown in the first row (*Classical Attack*) of Table 4.

Compared to classical cryptanalysis, the result of *Basic Attack* has no advantage over classical differential-linear cryptanalysis. For this, we design a new machine learning-aided differential-linear cryptanalysis using the generalized neutral bits.

5.1.2 Improved attack using neutral bits

This basic attack can be accelerated in various ways. Here, we focus on the following ideas:

1) In Gohr (2019); Bao et al. (2022), the neutral bits are used to aggregate enough samples following the same distribution. We use this to generate positive samples to improve the accuracy of the trained neural networks.
2) Considering that the neural networks are trained based on the positive sample generated by neutral bits, we construct the plaintext pairs used in the key-recovery attacks with neutral bits.

Before training neural networks and performing key-recovery attacks, the t probabilistic neutral bits $\{nb_1, ..., nb_t\}$ are found as follows:

Step 1. Given the 4-round differential trail $\Delta_I = 0x02110a04 \rightarrow \Delta_O = 0x81008102$, randomly generate 2^{32} plaintext pairs and obtain l conforming pairs in total ($\{(P_0, P_0 \oplus \Delta_I), ..., (P_l, P_l \oplus \Delta_I)\}$, denoted by CPs).

Step 2. Given $2^{16} - 1$ integers ranging from 1 to $2^{16} - 1$, save the elements whose hamming weight is no more than 3, denoted by I.

Step 3. For each candidate nb' from I, there are $N_{nb'}$ conforming pairs in $\{(P_0 \oplus nb', P_0 \oplus \Delta_I \oplus nb'), ..., \big(P_l \oplus nb', P_l \oplus \Delta_I \oplus nb'\big)\}$. The neutral probability of nb' is $\frac{N_{nb'}}{l}$.

Step 4. Sort these candidates by their neutral probability.

In the basic attack, one positive sample contains t random plaintext pairs. In the improved attack, given $t-1$ neutral bits $\{nb_1, ..., nb_{t-1}\}$ with high neutral probability and a random plaintext P_0, there are t plaintext pairs $\{(P_0, P_0 \oplus \Delta_I), (P_0 \oplus nb_1, P_0 \oplus \Delta_I \oplus nb_1), ..., (P_0 \oplus nb_{t-1}, P_0 \oplus \Delta_I \oplus nb_{t-1})\}$. And the t plaintext pairs are combined as a positive sample. Based on these positive samples, a new 8-round neural distinguisher is trained, whose accuracy is more than 70%.

Similarly, in the key-guessing phase, the neutral bits are used in the generation of plaintext pairs. Firstly, randomly generate N plaintexts $\{P_1, P_2, ..., P_N\}$. And for each plaintext P_i, the plaintext structure of P_i is generated by $\{(P_i, P_i \oplus \Delta_I), (P_i \oplus nb_1, P_i \oplus \Delta_I \oplus nb_1), ..., (P_i \oplus nb_{t-1}, P_i \oplus \Delta_I \oplus nb_{t-1})\}$. Encrypt each plaintext pair in the plaintext structure of P_i and obtain the ciphertext structure. In the key-recovery attacks, each ciphertext pair is decrypted using the guessed subkey, and the decrypted

Table 4 Summary of key-recovery attacks on 10-round SPECK32/64

Target Cipher	Configure	Type	a-bit advantage[1]			
			$a = 13$	$a = 12$	$a = 11$	$a = 10$
10−round SPECK32/64	$1 + 8 + 1$	Classical Attack	17.3%	30.5%	39.8%	53.4%
	$1 + 8 + 1$	Basic Attack	16.6%	28.5%	37.3%	50.0%
	$1 + 8 + 1$	Improved Attack	58.3%	79.6%	89.4%	96.4%

[1]The right subkey is expected to be in the first 2^{13-a} values according to the scores of candidates. That is $Index_{rk} \leqslant 2^{13-a}$, where $Index_{rk}$ is the index of the right subkey

Table 5 Results of key-recovery attacks on 11-round SPECK32/64

Target Cipher	Configure	Type	a-bit advantage[1]		
			$a = 15$	$a = 14$	$a = 13$
11−round SPECK32/64	$1 + 9 + 1$	Improved Attack	95.8%	99.4%	99.7%

[1]The right subkey is expected to be in the first 2^{13-a} values according to the scores of candidates. That is $Index_{rk} \leqslant 2^{13-a}$, where $Index_{rk}$ is the index of the right subkey

ciphertext pairs in the same ciphertext structure are put in one sample.

Due to the improvement in neural network training and the key-guessing phase, there are better results using $N = 2.56 \times 10^6$ plaintext pairs than classical differential-linear cryptanalysis. The results are shown in the penultimate row of Table 4.

5.2 Key recovery attack on 11-round Speck32/64

We train the neural networks $ND_9{}^{512}$ using $\Delta_I = 0x02110a04 \xrightarrow{9r} \Gamma_O = 0x58204020$ and 512 probabilistic neutral bits. The configuration of 11-round attacks is 1+9+1, *i.e.*, a free prepended invert round, a 9-round neural distinguisher, and a key guessing (last) round. Considering the locations of nonzero bits of Γ_O, we need to guess 15 bits of the final subkey ($K_8[0, 1, ..., 14]$). Then we perform the key-recovery attacks using $N \times t = 1.024 \times 10^7$ plaintext pairs. With 10^3 trials, the results are shown in Table 5.

6 Conclusion and future work

This paper mainly focuses on machine learning-aided differential-linear cryptanalysis. We design a valid machine learning-aided differential-linear cryptanalysis framework and apply it to round-reduced DES and round-reduced SPECK32/64. The result of differential-linear cryptanalysis indicates that machine learning-aided cryptanalysis can perform key-recovery tasks superior to orthodox methods, which is similar to differential-neural cryptanalysis. Our work indicates that machine learning can perform well not only in differential cryptanalysis but also in differential-linear cryptanalysis.

The successful application of our neural-assisted differential-linear cryptanalysis framework to block ciphers with distinct structural designs – including substitution-permutation networks (e.g., DES) and add-rotate-XOR ciphers (e.g., SPECK32/64) – demonstrates its broad applicability across various block ciphers. In fact, as shown in Section 3.1.1, the primary distinction between positive and negative samples lies in their adhesion to distinct Bernoulli distributions. Given the ciphertext pair (C, C') and the output mask Γ_O from the established classical cryptanalysis, the Bernoulli distribution characterizing positive samples is defined by the success probability of the event $\Gamma_O \cdot C \oplus \Gamma_O \cdot C' = 0$. The Bernoulli distribution characterizing negative samples is fixed as $Bern\left(\frac{1}{2}\right)$. Consequently, given a target block cipher, our neural-assisted differential-linear cryptanalysis framework is effective if the distribution characterizing positive samples is different from $Bern\left(\frac{1}{2}\right)$. Interestingly, the success probability of the event $\Gamma_O \cdot C \oplus \Gamma_O \cdot C' = 0$ is also the core of the classical differential-linear cryptanalysis. Consequently, we infer that the applicability domain of our neural-assisted framework aligns with that of classical differential-linear cryptanalysis.

While exhibiting identical theoretical applicability, our framework may empirically fail to match the round coverage achieved by classical differential-linear cryptanalysis. The main reason is as follows: (i) The success probability of the event $\Gamma_O \cdot C \oplus \Gamma_O \cdot C' = 0$ is close to $\frac{1}{2}$ if covering more rounds; (ii) Due to (i), the distributional divergence between positive and negative samples diminishes as round depth increases; (iii) Due to (ii), the trained network may be invalid in distinguishing the positive sample and negative sample when employing sample lengths identical to those used for shorter-round distinguishers; (iv) As shown in Table 2, the accuracy is higher if the single sample contains more units. Therefore, we can increase the sample length until the trained network is effective; (v) Unfortunately, computational constraints, particularly GPU memory limitations, impose an upper bound on feasible sample length extension in our experimental framework; (vi) Due to (v), we cannot generate enough training samples to train the distinguishers of 9-round DES and 10-round SPECK32/64 even if the success probability of the event $\Gamma_O \cdot C \oplus \Gamma_O \cdot C' = 0$ is unequal to $\frac{1}{2}$.

The principal limitations of our framework, along with potential avenues for enhancement, are delineated as follows:

1) Similar to the neural-differential cryptanalysis, the internal mechanisms of neural-based differential-linear distinguishers remain insufficiently analyzed. As shown in Section 3.1.1, leveraging statistically significant distributional differences (following the different Bernoulli distributions) observed between random data and ciphertext data, we engineer targeted training samples for neural distinguishers. However, whether there are undiscovered discriminative features remains a question requiring further investigation.

2) Analysis of the relationship between the length of the training sample and the performance of key recovery is lacking. As demonstrated in Table 2, the accuracy of the distinguisher is related to the length of the training sample. The accuracy of the distinguisher is also related to the performance of the distinguishing attack. Considering that the distinguishing attack is the cornerstone of the key recovery, the length of the training sample is related to the performance of the key recovery. In this work, we execute multiple key-recovery attacks for different training sample lengths. And the optimal length is selected through comparative analysis of complexity metrics. Consequently, investigating whether there is a quantifiable relationship between sample length and attack performance warrants research. Such characterization would enable optimal length selection without resource-intensive attack executions.
3) The training of neural-based differential-linear distinguishers relies on feature representations derived from classical differential-linear cryptanalysis. As shown in Section 3.1, the training samples are generated by leveraging the specified input difference Δ_I and the output mask Γ_O. For the neural-based differential-linear distinguishers of DES and SPECK32/64, we directly adopt the input difference and the output mask from the established classical cryptanalysis. Testing more input differences and output masks may lead to obtain a better distinguisher.
4) Due to the lack of computational resources, we only choose the same neural network architecture as Gohr's distinguishers to train our neural-based differential-linear distinguishers. For neural-differential distinguishers, empirical evidence confirms that state-of-the-art neural network architectures significantly enhance distinguishing accuracy. Given the documented efficacy gains in neural-differential distinguishers through architectural advancement, similar enhancements may potentially accrue to differential-linear variants when employing optimized network architectures.
5) Employing multiple output masks rather than a single mask may enhance our differential-linear distinguishers. In training the neural-differential distinguishers, it is found that only one input difference is used, and multiple output differences are used. In Sections 4 and 5, we train the distinguishers for DES and SPECK32/64 only using one output mask. Given a single output mask, one training sample unit is generated per ciphertext pair in Sections 4 and 5. The dimensionality of each positive sample corresponds directly to the number of ciphertext pairs used. When multiple output masks are utilized concurrently, multiple distinct units in one sample derive from individual ciphertext pairs. With this, an improved attack can be performed using fewer ciphertexts.

Supplementary Information The online version contains supplementary material available at https://doi.org/10.1007/s44443-025-00241-w.

Author Contributions All authors contributed to the study conception and design. Software, data collection and analysis were performed by Zezhou Hou and Jiongjiong Ren. The first draft of the manuscript was written by Zezhou Hou, Jiongjiong Ren and Shaozhen Chen. All authors commented on previous versions of the manuscript. All authors read and approved the final manuscript.

Funding This work is supported by the National Natural Science Foundation of China (No.62206312).

Availability of Data and Material The data underlying this paper are available in the paper. The supplementary code and data for this paper are available at github.com.

Declarations

Competing Interests The authors declare that there are no conflicts of interest.

Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by-nc-nd/4.0/.

References

Aljarf A, Zamzami H, Gutub A (2023) Integrating machine learning and features extraction for practical reliable color images steganalysis classification. Soft Comput 27(19):13877–13888. https://doi.org/10.1007/S00500-023-09042-7

Aljarf A, Zamzami H, Gutub A (2024) Is blind image steganalysis practical using feature-based classification? Multim Tools Appl 83(2):4579–4612. https://doi.org/10.1007/S11042-023-15682-6

Bacuieti N, Batina L, Picek S (2022) Deep neural networks aiding cryptanalysis: a case study of the speck distinguisher. In: Ateniese G, Venturi D (eds) Applied cryptography and network security - 20th international conference, ACNS 2022, Rome, Italy, June 20-23, 2022, Proceedings. Lecture Notes in Computer Science, vol 13269, pp 809–829. Springer, Cham (2022). https://doi.org/10.1007/978-3-031-09234-3_40

Bao Z, Guo J, Liu M, Ma L, Tu Y (2022) Enhancing differential-neural cryptanalysis. In: Agrawal S, Lin D (eds) Advances in Cryptology - ASIACRYPT 2022 - 28th international conference on the theory and application of cryptology and information security, Taipei, Taiwan, December 5-9, 2022, Proceedings, Part I. Lecture Notes in Computer Science, vol 13791, pp 318–347. Springer, Cham. https://doi.org/10.1007/978-3-031-22963-3_11

Bao Z, Lu J, Yao Y, Zhang L (2023) More insight on deep learning-aided cryptanalysis. In: Guo, J., Steinfeld R (eds) Advances in Cryptology - ASIACRYPT 2023 - 29th international conference on the theory and application of cryptology and information security, Guangzhou, China, December 4-8, 2023, Proceedings, Part III. Lecture Notes in Computer Science, vol 14440, pp 436–467. Springer, Cham. https://doi.org/10.1007/978-981-99-8727-6_15

Beaulieu R, Shors D, Smith J, Treatman-Clark S, Weeks B, Wingers L (2013) The SIMON and SPECK families of lightweight block ciphers. http://eprint.iacr.org/2013/404

Beierle C, Jean J, Kölbl S, Leander G, Moradi A, Peyrin T, Sasaki Y, Sasdrich P, Sim SM (2016) The SKINNY family of block ciphers and its low-latency variant MANTIS. In: Robshaw M, Katz J (eds) Advances in Cryptology - CRYPTO 2016 - 36th annual international cryptology conference, Santa Barbara, CA, USA, August 14-18, 2016, Proceedings, Part II. Lecture Notes in Computer Science, vol 9815, pp 123–153. Springer, Cham. https://doi.org/10.1007/978-3-662-53008-5_5

Bellini E, Gérault D, Hambitzer A, Rossi M (2023) A cipher-agnostic neural training pipeline with automated finding of good input differences. IACR Trans Symmetric Cryptol 2023(3):184–212. https://doi.org/10.46586/TOSC.V2023.I3.184-212

Benamira A, Gérault D, Peyrin T, Tan QQ (2021) A deeper look at machine learning-based cryptanalysis. In: Canteaut A, Standaert F (eds) Advances in Cryptology - EUROCRYPT 2021 - 40th annual international conference on the theory and applications of cryptographic techniques, Zagreb, Croatia, October 17-21, 2021, Proceedings, Part I. Lecture Notes in Computer Science, vol 12696, pp 805–835. Springer, Cham. https://doi.org/10.1007/978-3-030-77870-5_28

Biham E, Chen R (2004) Near-collisions of SHA-0. In: Franklin MK (ed) Advances in Cryptology - CRYPTO 2004, 24th annual international cryptology conference, Santa Barbara, California, USA, August 15-19, 2004, Proceedings. Lecture Notes in Computer Science, vol 3152, pp 290–305. Springer, Berlin, Heidelberg. https://doi.org/10.1007/978-3-540-28628-8_18

Biham E, Dunkelman O, Keller N (2002) Enhancing differential-linear cryptanalysis. In: Zheng Y (ed) Advances in Cryptology - ASIACRYPT 2002, 8th international conference on the theory and application of cryptology and information security, Queenstown, New Zealand, December 1-5, 2002, Proceedings. Lecture Notes in Computer Science, vol 2501, pp 254–266. Springer, Berlin, Heidelberg. https://doi.org/10.1007/3-540-36178-2_16

Chen Y, Shen Y, Yu H, Yuan S (2023) A new neural distinguisher considering features derived from multiple ciphertext pairs. Comput J 66(6):1419–1433. https://doi.org/10.1093/COMJNL/BXAC019

Deng H, Cao X, Cheng Y (2023) Attention in differential cryptanalysis on lightweight block cipher SPECK. In: 20th annual international conference on privacy, security and trust, PST 2023, Copenhagen, Denmark, August 21-23, 2023, pp 1–9. IEEE, Denmark. https://doi.org/10.1109/PST58708.2023.10320201

Dong T, Huang T (2020) Neural cryptography based on complex-valued neural network. IEEE Trans Neural Netw Learn Syst 31(11):4999–5004. https://doi.org/10.1109/TNNLS.2019.2955165

Farooqi N, Gutub A, Khozium MO (2019) Smart community challenges: enabling iot/m2m technology case study. Life Sci J 16(7):11–17. https://doi.org/10.7537/marslsj160719.03

Gohr A (2019) Improving attacks on round-reduced speck32/64 using deep learning. In: Boldyreva A, Micciancio D (eds) Advances in Cryptology - CRYPTO 2019 - 39th annual international cryptology conference, Santa Barbara, CA, USA, August 18-22, 2019, Proceedings, Part II. Lecture Notes in Computer Science, vol 11693, pp 150–179. Springer, Cham. https://doi.org/10.1007/978-3-030-26951-7_6

Gohr A, Leander G, Neumann P (2022) An assessment of differential-neural distinguishers. IACR Cryptol. ePrint Arch., p 1521. https://eprint.iacr.org/2022/1521

Greydanus S (2017) Learning the enigma with recurrent neural networks. arXiv:1708.07576

Gutub A, Shambour MK, Abu-Hashem MA (2023) Coronavirus impact on human feelings during 2021 hajj season via deep learning critical twitter analysis. J Eng Res 11(1):100001. https://doi.org/10.1016/j.jer.2023.100001

Gutub A, Altalhi S, Ghazwani B (2025) Offline efficient signature authentication using octave convolution neural network. Arab J Sci Eng 2025(1):1–16. https://doi.org/10.1007/s13369-025-09964-4

Hemalatha J, Mohan S, Kumar C, Gutub A, Sahu AK (2023) Towards improving the performance of blind image steganalyzer using third-order SPAM features and ensemble classifier. J Inf Secur Appl 76:103541. https://doi.org/10.1016/J.JISA.2023.103541

He K, Zhang X, Ren S, Sun J (2016) Deep residual learning for image recognition. In: 2016 IEEE conference on computer vision and pattern recognition, CVPR 2016, Las Vegas, NV, USA, June 27-30, 2016, pp 770–778. IEEE Computer Society, Los Alamitos, CA. https://doi.org/10.1109/CVPR.2016.90

Hou B, Li Y, Zhao H, Wu B (2020) Linear attack on round-reduced DES using deep learning. In: Chen L, Li N, Liang K, Schneider SA (eds) Computer security - ESORICS 2020 - 25th European symposium on research in computer security, ESORICS 2020, Guildford, UK, September 14-18, 2020, Proceedings, Part II. Lecture Notes in Computer Science, vol 12309, pp 131–145. Springer, Cham. https://doi.org/10.1007/978-3-030-59013-0_7

Hu J, Shen L, Sun G (2018) Squeeze-and-excitation networks. In: 2018 IEEE conference on computer vision and pattern recognition, CVPR 2018, Salt Lake City, UT, USA, June 18-22, 2018, pp 7132–7141. Computer Vision Foundation / IEEE Computer Society, Salt Lake City. https://doi.org/10.1109/CVPR.2018.00745

Korvel G, Kurowski A, Kostek B, Czyzewski A (2019) Speech analytics based on machine learning. In: Tsihrintzis GA, Sotiropoulos DN, Jain LC (eds) Machine learning paradigms - advances in data analytics. Intelligent systems reference library, vol 149, pp 129–157. Springer, Cham. https://doi.org/10.1007/978-3-319-94030-4_6

Langford SK, Hellman ME (1994) Differential-linear cryptanalysis. In: Desmedt Y (ed) Advances in Cryptology - CRYPTO '94, 14th annual international cryptology conference, Santa Barbara, California, USA, August 21-25, 1994, Proceedings. Lecture Notes in Computer Science, vol 839, pp 17–25. Springer, Berlin, Heidelberg. https://doi.org/10.1007/3-540-48658-5_3

Lu J, Liu G, Sun B, Li C, Liu L (2024) Improved (related-key) differential-based neural distinguishers for SIMON and SIMECK block ciphers. Comput J 67(2):537–547. https://doi.org/10.1093/COMJNL/BXAC195

Lyu L, Tu Y, Zhang Y (2022) Improving the deep-learning-based differential distinguisher and applications to simeck. In: 25th IEEE international conference on computer supported cooperative work in design, CSCWD 2022, Hangzhou, China, May 4-6, 2022, pp 465–470. IEEE, Piscataway, NJ. https://doi.org/10.1109/CSCWD54268.2022.9776036

Masure L, Dumas C, Prouff E (2020) A comprehensive study of deep learning for side-channel analysis. IACR Trans Cryptogr Hardw Embed Syst 1:348–375. https://doi.org/10.13154/TCHES.V2020.I1.348-375

Mou L, Meng Z, Yan R, Li G, Xu Y, Zhang L, Jin Z (2016) How transferable are neural networks in NLP applications? In: Su J, Carreras X, Duh K (eds) Proceedings of the 2016 conference on empirical methods in natural language processing, EMNLP 2016, Austin, Texas, USA, November 1-4, pp 479–489. The Association for Computational Linguistics, Texas. https://doi.org/10.18653/V1/D16-1046

Niu Z, Sun S, Liu Y, Li C (2022) Rotational differential-linear distinguishers of ARX ciphers with arbitrary output linear masks. In: Dodis Y, Shrimpton T (eds) Advances in Cryptology - CRYPTO 2022 - 42nd annual international cryptology conference, CRYPTO 2022, Santa Barbara, CA, USA, August 15-18, 2022, Proceedings, Part I. Lecture Notes in Computer Science, vol 13507, pp 3–32. Springer, Cham. https://doi.org/10.1007/978-3-031-15802-5_1

Picek S, Perin G, Mariot L, Wu L, Batina L (2023) Sok: deep learning-based physical side-channel analysis. ACM Comput Surv 55(11):227–122735. https://doi.org/10.1145/3569577

Rivest RL (1991) Cryptography and machine learning. In: Imai H, Rivest RL, Matsumoto T (eds) Advances in Cryptology - ASIACRYPT '91, international conference on the theory and applications of cryptology, Fujiyoshida, Japan, November 11-14, 1991, Proceedings. Lecture Notes in Computer Science, vol 739, pp 427–439. Springer, Berlin, Heidelberg. https://doi.org/10.1007/3-540-57332-1_36

Roy PK, Saumya S, Singh JP, Banerjee S, Gutub A (2023) Analysis of community question-answering issues via machine learning and deep learning: state-of-the-art review. CAAI Trans Intell Technol 8(1):95–117. https://doi.org/10.1049/CIT2.12081

Sarkar A, Bhattacharyya M, Garain U, Pal SK, Shantanu S, Bandyopadhyay S, R.Pal N (2024) Leveraging synergy to design neural differential distinguishers for lightweight block ciphers. IEEE Trans Emerg Top Comput Intell, pp 1–13. https://doi.org/10.1109/TETCI.2024.3485756

Shen D, Song Y, Lu Y, Long S, Tian S (2024) Neural differential distinguishers for GIFT-128 and ASCON. J Inf Secur Appl 82:103758. https://doi.org/10.1016/J.JISA.2024.103758

Standards NB (1977) Data Encryption Standard (DES), Final, Federal Information Processing Standard

Sun T, Shen D, Long S, Deng Q, Wang S (2022) Neural distinguishers on TinyJAMBU-128 and GIFT-64. In: Tanveer M, Agarwal S, Ozawa S, Ekbal A, Jatowt A (eds) Neural information processing - 29th international conference, ICONIP 2022, Virtual Event, November 22-26, 2022, Proceedings, Part V. Communications in Computer and Information Science, vol 1792, pp 419–431. Springer, Singapore. https://doi.org/10.1007/978-981-99-1642-9_36

Su H, Zhu X, Ming D (2020) Polytopic attack on round-reduced simon32/64 using deep learning. In: Wu, Y., Yung, M. (eds.) Information security and cryptology - 16th international conference, Inscrypt 2020, Guangzhou, China, December 11-14, 2020, Revised Selected Papers. Lecture Notes in Computer Science, vol 12612, pp 3–20. Springer, Cham. https://doi.org/10.1007/978-3-030-71852-7_1

Szegedy C, Liu W, Jia Y, Sermanet P, Reed SE, Anguelov D, Erhan D, Vanhoucke V, Rabinovich A (2015) Going deeper with convolutions. In: IEEE conference on computer vision and pattern recognition, CVPR 2015, Boston, MA, USA, June 7-12, 2015, pp 1–9. IEEE Computer Society, Boston. https://doi.org/10.1109/CVPR.2015.7298594

Wang G, Wang G (2021) Improved differential-ml distinguisher: machine learning based generic extension for differential analysis. In: Gao D, Li Q, Guan X, Liao X (eds) Information and communications security - 23rd international conference, ICICS 2021, Chongqing, China, November 19-21, 2021, Proceedings, Part II. Lecture Notes in Computer Science, vol 12919, pp 21–38. Springer, Cham. https://doi.org/10.1007/978-3-030-88052-1_2

Zahednejad B, Lyu L (2022) An improved integral distinguisher scheme based on neural networks. Int J Intell Syst 37(10):7584–7613. https://doi.org/10.1002/INT.22895

Zhang L, Wang Z, Chen Y (2023) Improving the accuracy of differential-neural distinguisher for des, Chaskey, and PRESENT. IEICE Trans Inf Syst 106(7):1240–1243. https://doi.org/10.1587/TRANSINF.2022EDL8094

Zhang L, Yao Y, Shi D, Chai D, Guo J, Wang Z (2025) Neural-inspired advances in integral cryptanalysis. CoRR. https://doi.org/10.48550/ARXIV.2505.10790. arXiv:2505.10790

Zou Z, Chen K, Shi Z, Guo Y, Ye J (2023) Object detection in 20 years: a survey. Proc IEEE 111(3):257–276. https://doi.org/10.1109/JPROC.2023.3238524

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.